Table des matières

Labo 05 OLAP — CTE et récursivité sur l'entrepôt 1
Partie 1 — De la sous-requête à la CTE 1
Partie 2 — Chaîner, générer, parcourir 2
Et ensuite ? 5

Labo 05 OLAP — CTE et récursivité sur l'entrepôt

Laboratoire du Cours 05, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

On met en pratique les CTE (`WITH`) et la récursivité sur l'entrepôt AdventureWorks DW, dans la ligne de commande DuckDB. Depuis `Projet_SQL/`, ouvrez l'entrepôt par `duckdb adventureworksdw.duckdb`, puis posez le schéma par `USE dbo;`. Comptez une demi-heure.

Les deux voies — Ce labo a un jumeau, `Labo_05_OLTP_SQL.md`, qui pose les mêmes questions sur le schéma normalisé d'AdventureWorks dans PostgreSQL. Faites celui-ci d'abord, puis l'autre, et gardez-les ouverts côte à côte : les écarts que vous verrez sont la vraie leçon du cours.

Une table à charger — Ce labo utilise `DimEmployee` (pour la hiérarchie) en plus des tables habituelles. Le script du Cours 00, `monter_entrepot_dw.sql`, la charge avec les sept autres ; relancez-le si votre entrepôt date d'avant. Vérifiez : `SELECT COUNT(*) FROM dbo.DimEmployee;` doit donner 296.

Partie 1 — De la sous-requête à la CTE

Théorie : `Theorie_05_SQL.md`, sections 1 et 2.

On reprend exactement la requête finale du Labo 04 (ventes mensuelles et variation), d'abord dans sa version imbriquée, puis nettoyée avec `WITH`. Tapez les deux : le résultat est identique.

Version imbriquée, à lire de l'intérieur vers l'extérieur :

```
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois) AS ventes_mois_prec
FROM (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
```


© 2026 iAlexMG. Tous droits réservés.

```
) AS mensuel
ORDER BY annee, mois;
```

Version CTE, à lire de haut en bas :

```
WITH mensuel AS (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
)
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois) AS ventes_mois_prec
FROM mensuel
ORDER BY annee, mois;
```

Observez. Même résultat, mais la seconde se lit dans l'ordre où on pense : je calcule `mensuel`, puis je m'en sers. Astuce de débogage : remplacez le `SELECT` final par `SELECT * FROM mensuel;` pour inspecter l'étape intermédiaire seule.

Partie 2 — Chaîner, générer, parcourir

2.1 — Chaîner deux CTE

Théorie : `Theorie_05_SQL.md`, section 3.

Objectif : les mois dont les ventes dépassent la moyenne mensuelle de l'entreprise.

```
WITH mensuel AS (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
),
moyenne AS (
    SELECT AVG(ventes) AS ventes_moyens FROM mensuel
)
SELECT m.annee, m.mois, m.ventes
FROM mensuel AS m
CROSS JOIN moyenne
WHERE m.ventes > moyenne.ventes_moyens
ORDER BY m.ventes DESC;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. La deuxième CTE (`moyenne`) part de la première (`mensuel`). Le `CROSS JOIN` colle l'unique valeur de moyenne à chaque mois, pour la comparaison.

2.2 — Générer une série, et combler les trous

Théorie : `Theorie_05_SQL.md`, sections 5 et 6.

D'abord la série pure, les douze premiers entiers :

```
WITH RECURSIVE nombres AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nombres WHERE n < 12
)
SELECT n FROM nombres;
```

Maintenant l'usage concret. En 2010, l'entreprise n'a vendu en ligne qu'en décembre. Un rapport mensuel brut ne montre donc qu'une seule ligne :

```
SELECT d.MonthNumberOfYear AS mois, SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
WHERE d.CalendarYear = 2010
GROUP BY d.MonthNumberOfYear;
```

On génère les 12 mois, puis on y raccroche les ventes par un `LEFT JOIN` : les mois sans vente réapparaissent à zéro.

```
WITH RECURSIVE mois AS (
    SELECT 1 AS m
    UNION ALL
    SELECT m + 1 FROM mois WHERE m < 12
),
ventes_2010 AS (
    SELECT d.MonthNumberOfYear AS mois, SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    WHERE d.CalendarYear = 2010
    GROUP BY d.MonthNumberOfYear
)
SELECT mois.m AS mois,
       COALESCE(v.ventes, 0) AS ventes
FROM mois
LEFT JOIN ventes_2010 AS v ON v.mois = mois.m
ORDER BY mois.m;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Les mois 1 à 11 affichent zéro, décembre porte le total réel. Sans la série générée, ces onze lignes n'existeraient tout simplement pas. C'est ainsi qu'on obtient un rapport sans trous.

2.3 — Remonter la hiérarchie (l'organigramme)

Théorie : `Theorie_05_SQL.md`, sections 5 et 7.

Objectif : dérouler tout l'organigramme depuis le PDG, avec le niveau de chacun.

```sql
WITH RECURSIVE organigramme AS (
    SELECT EmployeeKey, ParentEmployeeKey,
           FirstName || ' ' || LastName AS employe,
           1 AS niveau
    FROM dbo.DimEmployee
    WHERE ParentEmployeeKey IS NULL
    UNION ALL
    SELECT e.EmployeeKey, e.ParentEmployeeKey,
           e.FirstName || ' ' || e.LastName,
           o.niveau + 1
    FROM dbo.DimEmployee AS e
    JOIN organigramme    AS o ON e.ParentEmployeeKey = o.EmployeeKey
)
SELECT niveau, employe
FROM organigramme
ORDER BY niveau, employe;
```

Attendu : 296 lignes, réparties en 1, 9, 48, 210 et 28 par niveau.

Observez. Le niveau 1 ne contient qu'une personne (le PDG, seul sans manager). Chaque niveau suivant regroupe les employés rattachés au niveau du dessus. Changez l'ancre en `WHERE EmployeeKey = 18` (un manager intermédiaire) : vous n'obtenez plus que son équipe, lui compris.

Portabilité — Cette requête tourne telle quelle sur PostgreSQL (le `RECURSIVE` y est obligatoire, on l'a mis) et la concaténation `||` est standard. Ce qui changerait côté OLTP, c'est la table elle-même : le labo jumeau vous le montre.

2.4 — Compter les personnes, pas les lignes

Théorie : `Theorie_05_SQL.md`, section 9.

Objectif : voir que l'organigramme compte des versions, et le ramener aux personnes.

Le compte de 296 doit interpeller : AdventureWorks n'a pas 296 employés. Regardez d'abord d'où vient l'écart.

```sql
SELECT FirstName || ' ' || LastName AS employe,
       COUNT(*)                     AS versions,
```


© 2026 iAlexMG. Tous droits réservés.

```
       MIN(StartDate)                AS depuis,
       MAX(COALESCE(EndDate, DATE '9999-12-31')) AS jusqua
FROM dbo.DimEmployee
GROUP BY FirstName, LastName
HAVING COUNT(*) > 1
ORDER BY versions DESC, employe;
```

Attendu : 5 personnes, de deux à trois versions chacune, dont Sheela Word avec trois et Rob Walters avec deux.

`DimEmployee` est une dimension à évolution lente : un employé qui change de poste ou de supérieur n'écrase pas sa ligne, il en reçoit une nouvelle, et l'ancienne se ferme par une `EndDate`. La clé `EmployeeKey` identifie donc une version, pas une personne. La clé qui identifie la personne est la clé naturelle, `EmployeeNationalIDAlternateKey`, celle que le système source lui donne.

```
SELECT COUNT(*)                                      AS lignes,
       COUNT(DISTINCT EmployeeNationalIDAlternateKey) AS personnes,
       COUNT(*) FILTER (WHERE EndDate IS NOT NULL)    AS versions_closes
FROM dbo.DimEmployee;
```

Attendu : 296 lignes, 290 personnes, 6 versions closes.

Piège — filtrer l'état des deux côtés d'une récursion. La tentation est d'ajouter `WHERE Status = 'Current'` à l'ancre et à la partie récursive. Essayez : l'organigramme tombe à 254 personnes, et non à 290. La raison est que 23 employés bien courants sont rattachés à une version périmée de leur supérieur, par exemple les onze qui pendent sous l'ancienne ligne de Sheela Word. Le filtre coupe la branche au-dessus d'eux, et ils disparaissent avec elle. Une hiérarchie historisée se lit à une date donnée, en gardant les versions valides à cette date des deux côtés du lien, jamais en filtrant sur l'état courant. C'est le même piège qu'au labo transactionnel, en plus silencieux.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_05_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vos requêtes sont maintenant lisibles et vous savez répéter un calcul de proche en proche. Le Cours 06 attaque le SQL du reporting : sous-totaux, totaux généraux et tableaux croisés, avec `GROUPING SETS`, `ROLLUP`, `CUBE` et `PIVOT`.

© 2026 iAlexMG. Tous droits réservés.